



Front End Developer

Técnicas de Programação Avançada para a Web

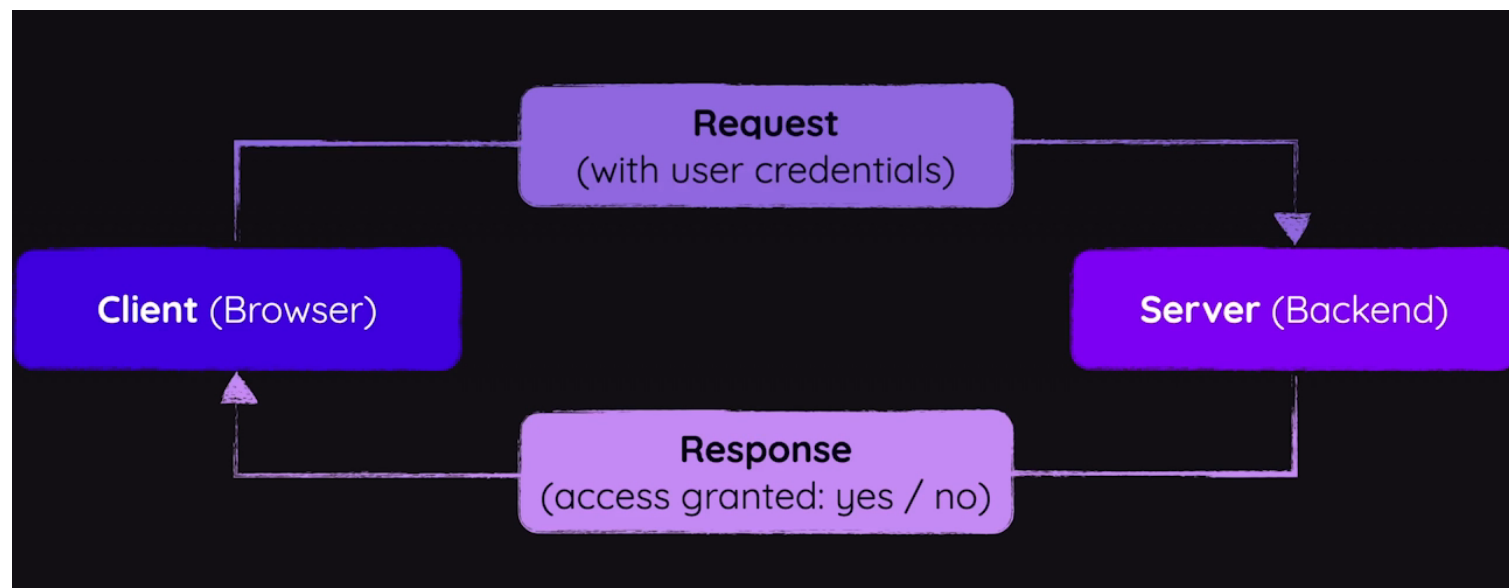
Sara Monteiro

sara.monteiro.prt@msft.cesae.pt

Autenticação – para que serve?

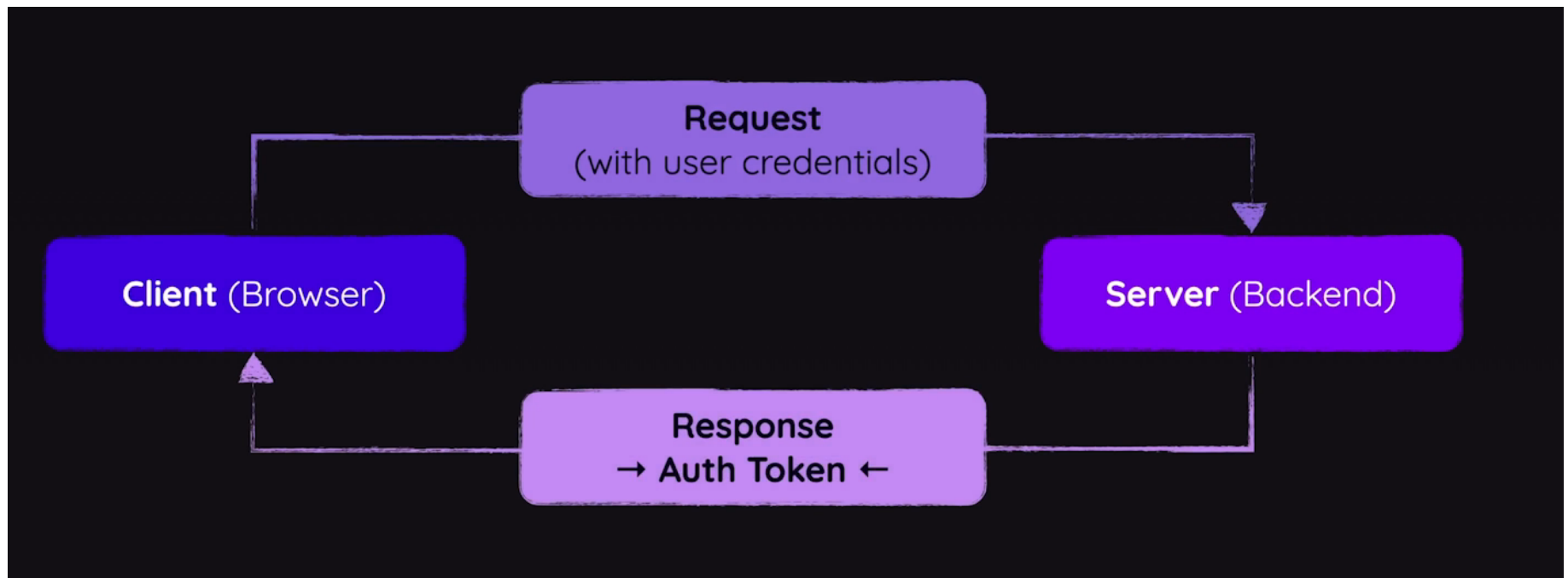
Iremos finalizar o estudo do módulo com as funcionalidades de Autenticação. Para que serve então?

➔ Para proteger conteúdo reservado a certos perfis de users



Autenticação: Authentication Tokens

- Os Tokens de Autenticação são uma informação que o BE nos envia a indicar que aquele user existe e que na nossa aplicação o podemos tratar como tal.



Autenticação: Authentication Tokens

```
import express from 'express';
import jwt from 'jsonwebtoken';

const app = express();

app.use(express.static('./images'));
app.use(bodyParser.json());

const SECRET_KEY = 'your_secret_key';

// Generate a JWT token
const createJSONToken = (email) => {
  return jwt.sign({ email }, SECRET_KEY, { expiresIn: '1h' });
};
```

Autenticação: Authentication Tokens

Em seguida enviamos os dados no Frontend para serem validados e caso ele retorne um user iremos ter essa informação na consola.

```
function handleSubmit(event){
  event.preventDefault();
  const authData = enteredValues;
  const response = fetch('http://localhost:3000/login', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(authData),
  })
  .then((response) => {
    if (response.ok) {
      console.log('Response is OK:', response.ok);
      return response.json();
    } else {
      console.error('Response is not OK:', response.status, response.statusText);
      throw new Error('Failed to authenticate');
    }
  })
  .then((data) => {
    console.log(data);
  });
}
```

Dados de sessão

De agora em diante, podemos proteger certos conteúdos de serem trazidos caso não haja token de validação. Para isso teremos que armazenar o nosso token numa variável de sessão que esteja disponível em todos os nossos componentes sem termos que importar/ exportar.

```
.then((data) => {  
  localStorage.setItem('token', data.token);  
});
```

```
console.log(localStorage.getItem('token'));
```

Autenticação: Logout

```
JSX Logout.jsx U X
src > pages > Auth > JSX Logout.jsx > Logout
1  import { useNavigate } from "react-router-dom";
2  import { useEffect } from "react";
3
4  export default function Logout(){
5      const navigate = useNavigate();
6      localStorage.clear();
7
8      useEffect(() => {
9          localStorage.clear();
10
11          navigate('/');
12      }, [navigate]);
13
14      return(
15          <div>
16              logging out
17          </div>
18      )
19  }
```

A função de logout limpa os dados todos de sessão e restabelece no user o estado inicial.

Actualização do Conteúdo de UI baseado na Autenticação

Agora que temos no localStorage um token, podemos usá-lo para esconder ou mostrar conteúdos na nossa aplicação. Por exemplo, o logout só faz sentido mostrar se o user estiver autenticado, e o login só deve aparecer quando não está autenticado.

Mas estes dados precisam de ser renderizados em toda a aplicação sempre que há uma mudança de rota. Para tal usaremos um loader.

Actualização do Conteúdo de UI baseado na Autenticação

```
const ROUTER = createBrowserRouter([
  {
    path: '/', element:<RootLayout/>,
    errorElement: <ErrorPage/>,
    id: 'root',
    loader: () => {
      return { login: localStorage.getItem('token') ? true :
        false };
    },
    children: [ You, last week • root Layout
```

Actualização do Conteúdo de UI baseado na Autenticação

```
function Header(){
  const loader = useRouteLoaderData('root');

  return (
    <header>
      <img src={reactLogo} alt="" />
      <h4>Aprendendo React!</h4>
      <div>
        <h1>As Minhas Funcionalidades</h1>
        <p><Link to="/shopping-list">Lista de Compras</Link></p>
        <p><Link to="/contacts">Contactos</Link></p>
        {loader.login &&
          <p><Link to="/logout">Logout</Link></p>
        }
        {!loader.login &&
          <p><Link to="/login">Login</Link></p>
        }
      </div>
    </header>
  );
}
```

Protecção de Rotas

Neste momento já sabemos proteger componentes ou conteúdos para utilizadores autenticados. E se quisermos proteger toda uma funcionalidade para um determinado role, por exemplo?

Ao fazer `console.log(data)` verificamos que contém o role do user, logo podemos adicionar no `localStorage`.

```
throw new Error('Failed to authenticate');
}
})
.then((data) => {
  localStorage.setItem('token', data.token);
  localStorage.setItem('role', data.role);
  navigate('/');
  console.log(data);
}).then((data) => {
  navigate('/');
});
```

Protecção de Rotas

Vamos então criar uma função que nos valide de acordo com o que queremos e aplicar a mesma nas rotas. Por exemplo, caso queira que a lista de compras apareça apenas a estudantes:

```
src > JSX ProtectedRule.jsx > ProtectedRoute

1  import React from 'react';
2  import { Navigate, Outlet } from 'react-router-dom';
3
4  export default function ProtectedRoute() {
5    const userRole = localStorage.getItem('role');
6
7    if (userRole !== 'student') {
8      return <Navigate to="/login" replace />;
9    }
10
11    return <Outlet />;
12  }
```

```
children: [
  {path: '/', element:<HomePage/>},
  {
    path: '/shopping-list',
    element: <ProtectedRoute />,
    children: [
      {
        path: '',
        element: <IndexShoppingList />,
      },
    ],
  },
],
```

Rotas com token de validação

Por segurança, hoje em dia muitas apis requerem validação de token para serem consultadas.

Caso isso aconteça a consultar APIs externas ou internas, no pedido deve ser enviado o token ao Backend.

```
export async function updatePlacesData(userPlaces){  
  const response = await fetch('http://localhost:3000/user-places',  
    {  
      method: 'PUT',  
      body: JSON.stringify({places: userPlaces}),  
      headers: {  
        "Content-Type": "application/json",  
        "Authorization": 'Bearer ' + token  
      }  
    })  
}
```

Recursos

- <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- <https://react.dev/>